

Assessment with Product Framing

Expectations

This exercise is intentionally designed to assess how you think, design, and build software in an AI-driven environment.

You are required to use AI tools to accelerate your work. We care about how you use them, the clarity of your thinking, and the quality of your engineering decisions.

We expect you to:

- Demonstrate clarity in thought and structured problem solving
- Show strong engineering fundamentals & product thinking
- Make thoughtful architectural and design decisions
- Write production-quality code and tests
- Use AI intentionally while maintaining correctness and quality

Please make incremental commits so we can understand how your solution evolved.

Goal:

Build a minimal yet usable salary management tool for an organization with 10,000 employees.

User Persona:

HR Manager of the org

Requirements:

1. Managing Employees -
 - a. Add, View, **update**, and delete employees via UI
 - b. The employee must have a full name, job title, country, salary, along with any other meaningful data that you believe should be captured
2. Salary Insights via UI -
 - a. Minimum, maximum, average salary of employees in a country
 - b. Average **salary** for the given Job Title in a country.
 - c. Any other meaningful metrics you believe are helpful for the user persona.

Technical Constraints:

You should build an end-to-end, fully functional software, including backend & UI.

1. Backend:

- a. {Language & Framework as per JD} (preferred) or any other framework of your choice. Relational database of your choice, like SQLite.
2. UI:
 - a. ReactJS or NextJS
 - b. Component library of your choice
3. Seeding:
 - a. Seed script with 10,000 employees. Full names should be generated by combining values from first_names.txt and last_names.txt
 - b. Assume that engineers run this script regularly, and performance of the script matters.
4. Readiness:
 - a. Fully functional deployed software
 - b. Any video demo of the software

Development Approach:

Using agentic AI tools of your choice, build the solution while maintaining high standards of code quality and test coverage.

Your solution should include:

- A meaningful set of unit tests that cover the core functionality
- Tests that are fast, deterministic, and easy to understand
- Good code structure, readability, and maintainability

Your commit history should show the evolution of your solution through incremental commits. This helps us understand how you approached the problem and how your implementation developed over time.

Artifacts

Along with your solution, please commit any artifacts that help us understand your thinking and approach.

Examples might include:

- Planning or design notes
- Architecture diagrams
- Prompts or instructions used with AI tools
- Trade-off explanations
- Performance considerations

There is no required format — use whatever helps best explain your approach.